

安全漏洞审计报告 · 第5轮

用户信息管理平台 路径遍历漏洞审计报告

User Management Platform — Path Traversal Vulnerability Assessment

目标系统：用户信息管理平台 (Flask)

报告日期：2026-07-13

漏洞等级：严重 × 2

修复状态：✅ 全部已修复

目录

1. 报告概述
2. 漏洞详情
3. 攻击演示（含实况结果）
4. 修复方案
5. 修复前后对照
6. 修复验证
7. 全部五轮汇总

1. 报告概述

1.1 本轮背景

系统新增了动态页面加载功能（`/page?name=help`），该功能读取 `pages/` 目录下的 HTML 文件并显示在首页上。由于代码直接将用户输入的 `name` 参数拼接到文件路径中，且没有做任何路径校验，导致严重的路径遍历漏洞。

1.2 什么是路径遍历？

路径遍历（**Path Traversal**）攻击者通过 `../` 跳出限制目录，读取服务器上的任意文件。这是 Web 应用最常见的文件操作漏洞之一。

2. 漏洞详情

VULN-19 : 动态页面路径遍历

● 严重

位置 : /page — GET 路由

漏洞代码 :

```
name = request.args.get("name", "")
page_path = os.path.join("pages", name) ← 无任何校验
open(page_path, "r").read()
```

描述 : 攻击者输入 ../app.py , 路径变为 pages/../app.py , 即 app.py。输入 ../../../../etc/passwd 可读取系统用户文件。

VULN-20 : 敏感文件泄露

● 严重

位置 : /page — GET 路由

描述 : 利用路径遍历可读取以下敏感文件 :

- app.py — Flask 源码泄露 (路由、数据库、密钥逻辑)
- data/users.db — SQLite 数据库文件 (所有用户数据)
- .git/config — Git 仓库配置泄露
- /etc/passwd — 系统用户列表
- /etc/shadow — 系统用户密码哈希

3. 攻击演示

3.1 读取 `app.py` 源码

攻击请求

GET /page?name=../app.py

攻击结果

成功读取 Flask 应用源码：

```
from flask import Flask, render_template
import sqlite3
app.secret_key = os.environ.get("SECRET_KEY", ...)
```

3.2 读取系统用户文件

攻击请求

GET /page?name=../../../../etc/passwd

攻击结果

成功读取到系统用户名列表：

```
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
```

3.3 读取密码哈希

攻击请求

GET /page?name=../../../../etc/shadow

攻击结果

成功读取到密码哈希（可用于暴力破解）：

```
root:$y$j9T$pl0EafxsTnm.o4cBGWzZL1$tZxW/pojXn31M1ALHNNkndLE/...
```

3.4 读取 Git 配置

攻击请求

GET /page?name=../.git/config

攻击结果

成功读取 Git 仓库配置：

```
[core]
repositoryformatversion = 0
url = https://github.com/sencly-book/user-management.git
```

4. 修复方案

核心修复：路径规范化 + 边界检查

```
# 1. 确定 pages 目录的绝对路径
pages_dir = os.path.join(os.path.dirname(__file__), "pages")

# 2. 将用户请求的路径解析为绝对路径
requested_path = os.path.abspath(os.path.join(pages_dir, name))

# 3. 检查是否在 pages 目录内
if not requested_path.startswith(os.path.abspath(pages_dir) + os.sep):
    page_content = "页面不存在" # 越界拒绝
```

原理：无论攻击者传入多少 `../`，`os.path.abspath()` 都会解析出真实路径。只要这个真实路径不在 `pages/` 目录下，就拒绝访问。

5. 修复前后对照

✖ 修复前

```
page_path = os.path.join("pages", name)
open(page_path).read()
```

- name = "../app.py"
- 读取 pages/../app.py
- ✔ 成功读取源码

✔ 修复后

```
requested_path = os.path.abspath(...)
if not in pages_dir: reject
```

- name = "../app.py"
- 规范化后不在 pages/ 下
- ❌ 拒绝访问

6. 修复验证

#	测试	结果
1	正常访问 ?name=help → 显示帮助中心	✓ 通过
2	路径遍历 ?name=../app.py → 被拦截	✓ 通过
3	路径遍历 ?name=../../etc/passwd → 被拦截	✓ 通过
4	路径遍历 ?name=../../etc/shadow → 被拦截	✓ 通过
5	路径遍历 ?name=../.git/config → 被拦截	✓ 通过
6	路径遍历 ?name=../data/users.db → 被拦截	✓ 通过

7. 全部五轮汇总

轮次	审计内容	漏洞数	严重	高危	中危	状态
第 1 轮	密码安全	8	4	-	4	✓
第 2 轮	SQL 注入	2	2	-	-	✓
第 3 轮	文件上传	1	1	-	-	✓
第 4 轮	业务逻辑	7	2	2	3	✓
第 5 轮	路径遍历	2	2	-	-	✓
合计		20	11	2	7	全部已修复

安全建议

- 永远不要信任用户输入的文件名：即使只是读取文件，也要校验路径
- 使用规范化路径：用 `os.path.abspath()` 解析后再做前缀检查
- 限定读取范围：确保程序只能访问预期目录内的文件
- 最小化文件权限：Web 应用不应具备读取系统文件（如 `/etc/shadow`）的权限